The Provenance Forest

Poker's generalisation step without enumerating refutations

Oruži Zarathustra Goertzel

7 November 2026

Abstract

Poker generalises an example by collecting, for every refutation of the example within the clause limit, the set of metasubstitutions the refutation uses. Vanilla enumerates these refutations one at a time, and their number grows exponentially with the clause limit. The provenance forest computes the same sets by dynamic programming: it solves each first-order goal once, for each exact size or, when hypothesis clauses are reused, as a bounded least fixpoint, and builds the sets of a refutation from those of its parts. It reads Vanilla's own problem and runs behind Poker's `generalise/3`. On the ten languages of Poker's AAAI-26 experiments, at the settings the paper reports and on its own sampled examples, its output is identical to Poker's in all 260 comparisons, and Poker's whole learner learns the same hypotheses with it; it is faster on nine languages, by 1.3 to 145 times in total, and slower on balanced parentheses (0.86). One forest run also contains the answer at every smaller clause limit and number of invented predicates, which turns the search for those settings into a single run plus look-ups. The same identity holds on Poker's other experiments, on tasks converted from other benchmarks and on 80 random grammar-learning problems.

1 The problem

Poker is the self-supervised learner of the Vanilla framework for meta-interpretive learning [3, 10], with Louise [4]; it learns from labelled and unlabelled examples and generates examples of its own [8]. Its generalisation step takes an example e, background knowledge B, a set of metarules, a signature (the target predicate and the invented predicates) and a clause limit K. Each refutation of e resolves goals either against B or against an instance of a metarule; each such instance is a *metasubstitution* σ, the metarule with its second-order variables bound. The *support* of a refutation is the set of metasubstitutions it uses. Poker's sub-hypotheses are the supports of the refutations of e with at most K steps.

Vanilla finds them by SLD-resolution with the metasubstitutions found so far as an argument of every call. Distinct refutations share most of their sub-refutations, but an untabled prover derives each shared part again in every refutation that contains it. Tabling the meta-interpreter does not recover the sharing: two calls on the same goal with different stores are different calls. On a^4b^4 at clause limit 7 Poker spends 5.13 billion inferences to produce 18 sets.

2 The decomposition

A refutation is a tree. Its root is a metasubstitution σ whose head matches the goal; its children are refutations of the body literals of σ; its leaves are background facts. Size and support are both

compositional:

$$\mathrm{size}(t) = 1 + \sum_i \mathrm{size}(t_i), \qquad \mathrm{supp}(t) = \{\sigma\} \cup \bigcup_i \mathrm{supp}(t_i),$$

with size 0 and support $\emptyset$ at a background leaf. The size counts steps with multiplicity, while the support is a set, as in Poker's store. Neither depends on the rest of the tree, so the family of supports of a goal's refutations of size exactly n,

$$F_n(g) = \{\, \mathrm{supp}(t) : t \text{ a refutation of } g,\ \mathrm{size}(t) = n \,\},$$

satisfies

$$F_0(g) = \{\emptyset\} \text{ if } g \in B, \text{ and } \emptyset \text{ otherwise,}$$
$$F_{n+1}(g) = \bigcup_{\sigma : g \leftarrow g_1, \ldots, g_m} \ \bigcup_{n_1 + \cdots + n_m = n} \big\{\, \{\sigma\} \cup S_1 \cup \cdots \cup S_m : S_i \in F_{n_i}(g_i) \,\big\},$$

and the generalisation of e at clause limit K is $\bigcup_{n=1}^{K} F_n(e)$. Each $F_n(g)$ is computed once and shared by every refutation that contains g at size n.

Theorem 1 (the recurrence characterises the supports)**.** *For a ground derivation forest (goals, background goals, and edges $g \leftarrow \sigma, [g_1, \ldots, g_m]$), the family F of supports of derivation trees satisfies the recurrences above, and every family satisfying them equals F.*

Proof sketch. That F satisfies the recurrences is the decomposition of a tree at its root. For uniqueness, let G satisfy them and show $G_n(g) = F_n(g)$ for all g by strong induction on n: at $n = 0$ both are given by B; at $n + 1$ both sides apply the same recurrence to families of sizes $n_i \leq n$, which agree by induction. □

Lemma 2 (pruning loses nothing)**.** *In the recurrence, a splitting $n_1 + \cdots + n_m = n$ contributes a set only if each body literal g_i has a derivation of size n_i. Hence restricting the splittings to those in which every literal is known to be derivable at its share keeps every set of $F_{n+1}(g)$.*

Both statements are machine-checked in Lean 4 with Mathlib, using no axioms beyond Lean's standard ones, together with positive and negative examples.

3 From ground forests to MIL goals

In meta-interpretive learning the forest is not given in advance. A goal is an encapsulated literal $P(a_1, \ldots, a_k)$ with some arguments bound; the body of Chain, $Q(X, Z), R(Z, Y)$, introduces an existential Z that is known only once $Q(x, Z)$ is solved. The forest therefore works in two passes over memo tables keyed by goal (up to variable renaming) and size.

Pass 1: instances. *reach*(g, n) is the set of instances of the goal g that have a refutation of size exactly n. A body is solved left to right: the size $n - 1$ is split among the literals, and the instances of each literal at its share bind the variables the next literals share with it. A literal of size 0 is answered by B; a literal of positive size is a target or invented predicate, answered by *reach* itself. A literal is memoised as its call pattern: when an argument before the last is bound, the last is left open, as the output of a grammar rule or a function, so that every call on the same input shares one entry. Solving a more general goal and unifying its answers with the literal gives exactly the literal's answers, so the pattern affects only the cost.

Pass 2: supports. $fam(i, n)$ is $F_n(i)$ for an instance i, built by the recurrence, but only over the splittings pass 1 found feasible. By the pruning lemma this loses nothing.

Invented predicates. Each memo key also carries which invented predicates are available: the i-th invented predicate may head a clause only once the $(i-1)$-th is available. This names inventions canonically, so that supports differing only by a renaming of invented predicates are not produced twice. A goal of the ground forest is then a goal together with this state, and the theorem applies unchanged.

Metarule constraints. Poker's constraints, such as $P \neq Q$ in Chain, are tests on a single metasubstitution. The forest calls Vanilla's own check on each metasubstitution when it is built and again as its body literals bind its symbols; the constraints compare symbols with `==`, so that a symbol not yet bound passes. The check depends only on the metasubstitution's instance, and is tabled on it. A constraint on one metasubstitution removes edges from the forest and does not interact with the decomposition.

4 Behind Poker's API

The forest reads a Vanilla MIL problem as Poker does: the encapsulated metarules, the background knowledge Vanilla writes to the `user` module, the configuration's metarule constraints, and Poker's signature of the target and its invented predicates. `forest_generalise/3` answers as Poker's `generalise/3`: it computes each positive example's sub-hypotheses with the forest and passes them through Poker's own constraint check, sorting, unfolding and renaming of invented predicates, so the two outputs can be compared as Prolog terms. `use_forest/0` wraps `generalise/3`, so Poker's `learn/1` runs on the forest unchanged; it falls back to Vanilla wherever the forest does not apply.

The forest applies wherever Poker keeps every proof and generalises examples one at a time without threads. Some settings let Vanilla resolve goals against background knowledge and metarules only (`fetch_clauses` without `hypothesis`): $a^n b^n$ and parity in Vanilla's examples, and the Hilbert curve and Sierpinski triangle of the AAAI-26 experiments. There every step instantiates a metarule and the clause limit bounds the steps. Most others, hello world and the six context-free languages of the AAAI-26 experiments among them, use Vanilla's default, which also resolves goals against hypothesis clauses already built. There the clause limit bounds the distinct metasubstitutions of a refutation, not its steps, and size is no longer additive; the sub-hypotheses without reuse are a strict subset of Poker's (hello world: 16 of 18; the Chomsky–Greibach parentheses: 64 of 256).

With reuse. The forest then computes the least fixpoint of the same recurrence without sizes: a goal's supports are the sets $\{\sigma\} \cup S_1 \cup \cdots \cup S_m$ of at most K metasubstitutions, over clauses $\sigma : g \leftarrow g_1, \ldots, g_m$ and supports S_i of the g_i, and of background facts the empty set. SWI-Prolog's tabling [12] computes it, cycles included, each literal tabled as its call pattern; it is finite because supports are bounded sets over finitely many metasubstitutions. A body literal whose predicate symbol is still open (the Q and R of Chain) is answered by the background knowledge, or has its symbol chosen first among the target and invented predicates, so that the constraints are checked once per choice rather than once per support; the answers are those of the open goal. The invention state is threaded in Vanilla's depth-first order as before: a clause headed by the i-th invented predicate is introduced only once the $(i-1)$-th is available, and a clause met again passes, its head already available. A check on the set alone (that the inventions used are inv_1 to inv_j) is too weak: it admits 342 sub-hypotheses for the parentheses where Poker has 256.

Theorem 3 (the bounded fixpoint characterises the supports with reuse)**.** *For a ground derivation forest and a bound K, the least family closed under the recurrence without sizes, every support and every partial union of a body kept within K labels, is exactly the set of supports of derivation trees with at most K labels; a label may occur any number of times in a tree.*

The proof is by induction both ways: a subtree's support is part of its tree's, so the bound prunes no tree whose support stays within it. It is machine-checked in Lean 4 with Mathlib, with Lean's standard axioms only, beside the theorem of Section 2; that the depth-first invention state makes the forest's items correspond to Vanilla's refutations is checked against Poker, not proved.

5 Why-provenance

$F_n(g)$ is the why-provenance of g [1], restricted to derivations of size n, with metasubstitutions in the place of database tuples. In the semiring view of provenance [2] its values are sets of sets, added by union over alternative derivations and multiplied by pairwise union over the literals of a body. Grading by size keeps the family finite for recursive targets, and the theorem above is the statement that the graded fixpoint is the provenance of the derivation trees.

6 Cost

The memo tables hold one entry per goal, size and invention state. Pass 1 costs, per entry, one step for each metarule, choice of symbol, splitting of the size and intermediate endpoint. Pass 2 builds only sets that are parts of sub-hypotheses: every set it builds is contained in one of the output's sets. Poker's cost grows with the number of refutations instead, and of the partial ones that fail, a number that grows exponentially with K even when the number of distinct supports does not: at a^4b^4, limit 7, there are 18 supports.

With reuse there are no sizes: a goal's family holds every support within the clause limit, whether or not the rest of a refutation can still fit beside it. Vanilla's search carries the metasubstitutions found so far and stops as soon as they exceed the limit, so where few metasubstitutions are allowed and the strings are long, the budget prunes its search more than sharing saves the forest (Section 8).

7 Methodology

What is compared. Three things are compared, each on identical inputs.

1. *The generalisation step.* Poker's `generalise/3` and `forest_generalise/3` receive the same encapsulated positive examples and metarules, with the background knowledge written to the `user` module as Poker's `top_program/7` writes it. Their outputs, the sub-hypotheses after Poker's own constraint check, sorting, unfolding and renaming of invented predicates, are compared as Prolog terms up to variable renaming (`=@=`). When no example has a refutation within the clause limit both fail, through the same post-processing; that outcome counts as agreement. An exception in either (a resource limit, a time limit) is recorded as such and is never counted as agreement.

2. *The whole learner.* Poker's `learn/7` (generalise, generate examples, label, specialise) runs once with Vanilla's generalisation and once with the forest's, through `use_forest/0`. The learned hypotheses and the labellings (the examples labelled positive and negative) are compared as terms, and each hypothesis is evaluated by the experiment's own test harness.

3. *Smaller limits read off one run* (Section 8.2). The read-off at each smaller clause limit and `max_invented` is compared, as terms, with the forest run directly there and with Poker run directly there.

Benchmarks.

- The experiments of Poker's AAAI-26 paper [8]: six context-free languages over $\{0, 1\}$ (even parity, $a^n b^n$, equal numbers of a and b in any order, $a^n b^m$ with $n \geq m \geq 0$, palindromes, balanced parentheses) under weak Chomsky–Greibach normal-form metarules, and four L-systems (the Dragon, Hilbert and Koch curves and the Sierpinski triangle, from Prusinkiewicz and Lindenmayer [11]) under Lindenmayer normal-form metarules. An L-system example relates a string to the one its productions rewrite it to. The paper's code appendix [9] is run as published; nothing of it is copied into Vanilla's tree.
- Poker's own experiments in Vanilla [3]: $a^n b^n$, parity, hello world, and the Chomsky–Greibach settings of its experiment script.
- Task files written for Poker's setting from other benchmarks: Delétang et al. [5], Popper's suite [6], Hopper's [7], Louise's even and odd [3, 4] and an Inverse probe. Each file's header states its source, the version taken and the conversion; they are in `data/forest_examples/benchmarks/survey`.
- 80 random grammar-learning problems (Section 8).

Settings. For the AAAI-26 experiments every setting is read from the code appendix itself rather than transcribed: each language's options from its `set_configs/2` clause (clause limit, `max_invented`, the unfolding of invented predicates, hypothesis reuse and tabling), and its labelled examples, numbers of generated examples and testing examples from the arguments its range experiment passes to `setup_range_experiments/12`, captured before any experiment runs (Table 1). These are the settings the paper recommends and reports, and they lead the results. Larger clause limits elsewhere in this note (Table 6) are scaling tests of the generalisation step, not recommended settings: with hypothesis reuse Poker learns $a^n b^n$ at clause limit 3 with one invented predicate, while without reuse a refutation of $a^n b^n$ takes $2n - 1$ steps, so the limit must grow with the example. In Poker's design a limit beyond what the target needs makes the refutation search exhaust its resources, and the right limits are found by raising the clause limit and `max_invented` from $(1, 0)$ until a correct hypothesis appears; Section 8.2 measures that procedure.

Examples and seeds. Labelled examples are generated by the appendix's own `generate_initial/3` from each iteration of the range experiment, which samples them at random from the strings of the language within a length range. Before each problem the random generator is seeded with a fixed seed s: seeds 1 to 5 for the generalisation step, 1 to 3 for the whole learner. The number l of labelled examples reported is the number generated: where an iteration asks for more than the strings of its length range, the generator returns fewer. Poker's generalisation draws one random number per refutation (a Bernoulli trial of probability 1, which keeps every refutation); the forest draws none. So that the steps after generalisation (the order of the generated examples) see the same random numbers, the generator is reseeded with $s + 1000003$ after `generalise/3` with either learner, and with $s + 2000003$ before the testing examples are sampled.

Time. Times are the CPU time of the SWI-Prolog process, `statistics(cputime)`, around one call, after a garbage collection; wall-clock times and inference counts were recorded too. The generalisation step's time includes Poker's post-processing for both learners; Vanilla's tables are refreshed before Poker's call, outside the timing, as `top_program/7` does. Each comparison runs Poker then the forest in one process, one problem after another. Every generalisation is limited to 600 s and every `learn/7` to 1200 s. Times are single runs per seed; the tables give medians over seeds and totals over all comparisons, and a ratio is Poker's time over the forest's. The benchmark ran one process at a time, except the generalisation step of the Hilbert curve and the Sierpinski triangle, measured while two other benchmark processes ran. The machine was shared with other work, so times are indicative and the ratios, measured side by side, more reliable than the absolute values.

Software. SWI-Prolog 10.1.9 on x86-64 Linux. The appendix runs in a working copy that differs from it only in the two lines its README tells the user to edit: Poker as the learner, and the experiment script. Its Vanilla and Poker sources are byte-identical to this fork's, apart from the selected experiment file and one auxiliary predicate the appendix adds; the forest is loaded from this fork. The scripts are in `data/forest_examples/benchmarks/aaai26`: `setup_aaai26.sh` makes the working copies, `aaai26_bench.pl` runs the comparisons, `run_aaai26.sh` runs the protocols reported here and `summarise.pl` tabulates them.

8 Results

8.1 The AAAI-26 experiments at their own settings

Table 1 lists the settings and Table 2 the generalisation step. In all 260 comparisons the two outputs agree: 248 are identical lists of sub-hypotheses, and in the other 12 neither learner finds a refutation. The forest is faster on nine of the ten languages, most on those whose refutations are long or branch widely (palindromes, by 145 times in total; the Koch curve and even parity, 29 times; the Hilbert curve, whose refutations without reuse take up to 8 steps, 18 times). It gains least on $a^n b^n$ (1.29) and is slower on balanced parentheses (0.86). Both reuse hypothesis clauses at small clause limits, $a^n b^n$ on strings of up to 44 symbols and the parentheses with two invented predicates. Vanilla's search, which carries the metasubstitutions found so far, is cut early by the clause limit, while the forest computes every support of every intermediate goal within the limit, whether or not the rest of the refutation can still fit.

The whole learner. Poker's `learn/7` ran with Vanilla's generalisation and with the forest's on the first and the last iteration of each range, with the least and the greatest number of generated examples of the range, seeds 1 to 3: 12 pairs of runs per language (Table 3). In all 120 pairs the learned hypotheses and the labellings are identical. At the last iteration with the most generated examples Poker learns a correct hypothesis (one that accepts every positive and rejects every negative testing example) in the runs counted in the table, the same with either generalisation. The steps after generalisation are Vanilla's in both runs and cost the same; where they dominate, as on the L-systems with 1,500 generated examples, the forest saves only the generalisation's share.

8.2 Choosing the limits

The paper's settings are found by raising the clause limit K and `max_invented` M from $(1, 0)$ until Poker learns a correct hypothesis. Every candidate setting needs its own refutation search with

Language	Metarules	K	M	Reuse	Unfold	Labelled l	Generated k	Testing (pos. / neg.)
even parity	I, C	5	1	yes	none	1–21, len. 0–5	0–5	all, 6–10 / all, 0–5
$a^n b^n$	I, C	3	1	yes	all	1–21, len. 0–45	0–5	all, 46–80 / all, 0–12
equal a, b	I, C	4	2	yes	all	1–21, len. 0–6	0–250	all, 7–10 / all, 0–8
$a^n b^m$	I, C	4	1	yes	learned	1–41, len. 0–9	0–5	all, 10–18 / all, 0–13
palindrome	I, C, T	5	0	yes	learned	1–21, len. 0–5	0–250	all, 6–16 / all, 0–10
parentheses	I, C	4	2	yes	all	1–21, len. 0–10	0–10	1500, 11–21 / 1500, 0–15
Dragon curve	LNF	5	2	yes	all	1–41, len. 0–4	0–1500	all, 5–10 / all, 0–4
Hilbert curve	LNF	8	6	no	all	1–21 + 1–21	0–1500	1500, 0–12 / 1500, 0–4
Koch curve	LNF	6	4	yes	all	0–20 + 1–5	0–1500	1500, 10–14 / 1500, 0–5
Sierpinski triangle	LNF	6	5	no	all	1–31 + 1–126	0–1500	1000, 0–14 / 1000, 0–5

Table 1: The settings of the AAAI-26 experiments, as read from the code appendix. K: clause limit; M: `max_invented`; reuse: Vanilla resolves goals against hypothesis clauses too (and is tabled); unfold: Poker's unfolding of invented predicates. Metarules: Identity, Chain, Tri-Chain under weak Chomsky–Greibach constraints, or Lindenmayer normal form. Labelled: the numbers requested over the range's iterations and the string lengths sampled from (for the L-systems, a second set of strings long enough to contain the system's variables: Hilbert 11–13, Koch 8–9, Sierpinski 9–10). Testing: how many strings (*all*, or a sample) and their lengths.

Vanilla. One run of the forest at (K, M) contains the generalisation at every smaller (K', M') with $K' \geq 2$, and `forest_family/3` with `forest_readoff/4` reads it off:

- *Without reuse* the memo tables are indexed by exact size: the supports at K' are those of the grades up to K'.
- *With reuse*, by the bounded-fixpoint theorem, the fixpoint at K' and the supports of at most K' metasubstitutions in the fixpoint at K are both the supports of the derivation trees with at most K' labels.
- *Invented predicates.* A refutation that names only the first M' invented predicates is a refutation under the smaller signature, and conversely: Vanilla introduces invented predicates in order, and resolves every literal of an invented predicate against a clause that predicate heads. The supports at M' are those that name no later invented predicate. The invention state is capped by the signature's size, but a refutation over the first M' invented predicates passes through the same states under both signatures.
- *At $K' = 1$ the problem changes*: there Poker's `top_program/7` also writes the labelled examples as background knowledge, so that a single clause can resolve a recursive literal against another example. The read-off does not apply, and the forest runs directly.

The read-off also leaves SWI-Prolog's tables as `generalise/3` leaves them, all abolished: Poker's later steps start from no tables, and a first version of the read-off that skipped this learned a different hypothesis at the next candidate.

The deepening. On each language's calibration experiment (the single experiments the code appendix provides for calibrating the learners' settings, with the number of generated examples its `set_configs/2` sets) the deepening tries (K, M) for $K = 1, 2, \ldots$ and, for each, M from 0 to $K - 1$ (a hypothesis of K clauses defines at most $K - 1$ invented predicates), and stops at the first whose learned hypothesis is correct on the experiment's testing examples, or at a time limit of 300 s

Language	Agree	Sub-hyps.	Last iteration		All comparisons		Poker / forest	
			Poker	Forest	Poker	Forest	total	range
even parity	25/25	32–631	5.83 s	233 ms	81.9 s	2.79 s	29.4	22–60
$a^n b^n$	25/25	4–9	1.69 s	1.32 s	22.1 s	17.2 s	1.29	1.07–5.9
equal a, b	25/25	29–271	2.55 s	1.13 s	35.6 s	15.8 s	2.26	1.95–2.66
$a^n b^m$	25/25	24–105	1.43 s	174 ms	24.6 s	2.81 s	8.75	7.5–23
palindrome	30/30	3–124	11.8 s	86 ms	196.5 s	1.36 s	145	65–391
parentheses	25/25	16–107	3.71 s	4.68 s	51.6 s	59.9 s	0.86	0.75–1.98
Dragon curve	25/25	2–46	3.28 s	175 ms	43.0 s	2.01 s	21.4	15–55
Hilbert curve	25/25	4–136	63.0 s	3.46 s	919.5 s	49.9 s	18.4	8.6–22
Koch curve	25/25	1–17	6.35 s	274 ms	83.6 s	2.84 s	29.5	15–44
Sierpinski triangle	30/30*	1–30	43.0 s	5.31 s	683.8 s	88.7 s	7.71	7.0–8.9

Table 2: The generalisation step on the AAAI-26 experiments at their own settings (Table 1): every iteration of each range experiment, seeds 1 to 5. Agree: comparisons whose outputs are identical as terms. Sub-hyps.: the number of sub-hypotheses Poker's `generalise/3` returns. Times are CPU times, medians over the seeds at the last iteration and totals over all comparisons; the last iteration has the most labelled examples (l = 21, 21, 21, 17, 21, 21, 41, 42, 25 and 157). Range: the least and greatest ratio over the comparisons. *In 12 of the 30, mostly at the first iterations, no example has a refutation within the clause limit and both learners return none.

per `learn/7`. It runs three ways: with Poker; with the forest run at each candidate; and with the read-off, which runs the forest once at $(K, K-1)$ when the deepening first reaches clause limit K and reads off the candidates up to it. Table 5 gives the results for seed 1. The deepening stops at the paper's own settings for five languages (even parity, $a^n b^n$, $a^n b^m$, the parentheses and the Koch curve), and earlier for two: the palindromes at $(2, 0)$ and the Dragon curve at $(3, 1)$, where the learned grammars are the correct ones (the clause limit bounds each sub-hypothesis, and the learned program is the union of those that survive specialisation). For equal numbers of a and b no candidate up to $(5, 4)$ gives a correct hypothesis on the calibration experiment; there Poker stops at $(5, 3)$ on a table-space limit, the forest run at each candidate on the time limit (its labelling alone takes about 285 s), and the read-off completes all 15 candidates. At every candidate that two deepenings both reach, they learn the same hypothesis. In these runs the steps after generalisation take a large share of the time, so the forest saves the generalisation's share: from 1.2 to 22 times on it. The read-off costs about as much as the forest run at each candidate here, because at each clause limit it runs the forest with the largest number of invented predicates, which costs most with reuse; it pays off when one run serves many candidates, as on the Hilbert curve in Table 4.

8.3 Poker's own experiments: scaling and reuse

Table 6 is a scaling test of the generalisation step, not a recommended setting. Poker's $a^n b^n$ experiment in Vanilla does not reuse hypothesis clauses, so a refutation of $a^n b^n$ takes $2n - 1$ steps and the clause limit must grow with the example. Poker's cost grows with the number of refutations, exponentially in the clause limit, while the number of distinct supports need not grow (18 at $a^4 b^4$, limit 7). With reuse, as in the AAAI-26 setting, the same language is learned at clause limit 3 (Tables 2 and 5). At limits too small to fit any refutation neither learner has an answer: Poker's `generalise/3` fails, and so does the forest's, through the same post-processing.

On 80 random grammar-learning problems (terminals, metarules among Identity, Chain and Tri-chain, constraints, one example string of length 0 to 5, clause limits 1 to 4, 0 to 2 invented

Language	Same	`learn/7`		of which generalisation		Correct
		Vanilla	Forest	Vanilla	Forest	
even parity	12/12	7.16 s	389 ms	6.56 s	104 ms	3/3
$a^n b^n$	12/12	2.02 s	1.48 s	1.89 s	1.41 s	0/3
equal a, b	12/12	13.1 s	11.6 s	2.88 s	1.37 s	2/3
$a^n b^m$	12/12	1.81 s	388 ms	1.66 s	299 ms	0/3
palindrome	12/12	12.2 s	1.75 s	9.88 s	78 ms	3/3
parentheses	12/12	4.30 s	5.42 s	4.25 s	5.35 s	1/3
Dragon curve	12/12	25.9 s	21.2 s	3.94 s	511 ms	3/3
Hilbert curve	12/12	154 s	87.8 s	62.7 s	3.56 s	3/3
Koch curve	12/12	13.0 s	6.85 s	6.35 s	126 ms	3/3
Sierpinski triangle	12/12	51.0 s	17.6 s	39.9 s	5.05 s	1/3

Table 3: Poker's `learn/7` with Vanilla's generalisation and with the forest's. Same: pairs of runs (iterations 1 and last, the least and greatest number of generated examples, seeds 1 to 3) whose hypotheses and labellings are identical. The times are medians over the seeds at the last iteration with the greatest number of generated examples (5, 5, 250, 5, 250, 10, 1500, 1500, 1500, 1500); Correct: runs of those three whose hypothesis is correct on the experiment's testing examples, with either generalisation.

predicates, no reuse) the two outputs are identical in all 80. Table 7 gives the settings of Poker's Chomsky–Greibach script and its hello world, which reuse hypothesis clauses.

8.4 Other benchmarks

Table 8 goes beyond Poker's experiments, to the task files of Section 7, each with its own clause limit and without reuse. The outputs are identical on the 13 tasks Poker finishes within 200 s. Two of them (reverse string and parity check without normal-form constraints) need a large stack: with SWI-Prolog's default, Poker's search raises a resource error, which the comparison now reports as an exception rather than as a failure of both. On the other three Poker does not finish; the forest's supports alone, from an earlier run, are given. The forest is slower than Poker only on list membership, where Poker's search is shallow and the forest's memo tables cost more than they save.

8.5 The same algorithm in C

CeTTa, a MeTTa runtime, has a learner built on the same recurrences, with the supports computed by a provenance kernel written in C. It takes the problem grounded into MeTTa, which needs the background knowledge to be facts, as it is for the six context-free languages. On iterations 1, 3 and the last of each of their ranges, seeds 1 to 3 (54 problems, 1,068 examples), its supports equal the SWI forest's, as sets, for every example. Table 9 gives its times: they are of the supports alone, without Poker's post-processing, and do not include grounding the problem. Where the SWI forest is slowest, on $a^n b^n$ and the parentheses, the C kernel is 7 and 58 times faster than it at the last iteration, and 9 and 47 times faster than Poker: the SWI forest's cost there lies in its representation (tabled answers and list unions) more than in the recurrence.

Language	Ceiling	Cells	Agree	One run + read-offs	Forest at each	Poker at each	$K' = 1$
even parity	(5, 1)	32	32	0.12 s	0.14 s	6.27 s	differs
$a^n b^n$	(3, 1)	16	16	0.79 s	0.90 s	1.52 s	differs
equal a, b	(4, 2)	36	36	1.60 s	1.73 s	4.48 s	differs
$a^n b^m$	(4, 1)	24	24	0.24 s	0.31 s	3.38 s	differs
palindrome	(5, 0)	16	16	0.03 s	0.06 s	12.3 s	differs
parentheses	(4, 2)	36	36	4.45 s	5.25 s	7.25 s	differs
Dragon curve	(5, 2)	48	48	0.11 s	0.29 s	7.20 s	differs
Hilbert curve	(8, 6)	196	196	2.92 s	16.8 s	223 s	same
Koch curve	(6, 4)	100	100	0.14 s	0.79 s	20.3 s	differs
Sierpinski triangle	(6, 5)	120	120	14.1 s	68.8 s	545 s	3 of 4

Table 4: One forest run at each language's own limits (the ceiling), read off at every clause limit K' from 2 to the ceiling's and every `max_invented` M' from 0 to its, on iterations 1 and 2 of the range with seeds 1 and 2 (for the Sierpinski triangle iterations 3 and 6, the first ones whose examples have refutations). Agree: cells where the read-off, the forest run directly at (K', M') and Poker run directly there all give the same output. The times are totals over the cells: the four runs at the ceiling with all the read-offs, the forest run at every cell, and Poker at every cell. $K' = 1$: whether the ceiling run's supports of one step equal the forest's own at clause limit 1, where the labelled examples are background knowledge too (in the four problems of each language).

Language	Paper	Found	Tried	Poker		Forest at each		Read-off	
				total	gen.	total	gen.	total	gen.
even parity	(5, 1)	(5, 1)	12	5.59 s	5.06 s	628 ms	227 ms	631 ms	229 ms
$a^n b^n$	(3, 1)	(3, 1)	5	124 ms	68 ms	82 ms	28 ms	88 ms	33 ms
equal a, b	(4, 2)	none	14–15	429 s[a]	39.9 s	708 s[b]	21.5 s	983 s	26.7 s
$a^n b^m$	(4, 1)	(4, 1)	8	465 ms	377 ms	122 ms	48 ms	145 ms	62 ms
palindrome	(5, 0)	(2, 0)	2	1.19 s	86 ms	1.17 s	59 ms	1.20 s	72 ms
parentheses	(4, 2)	(4, 2)	9	3.02 s	2.71 s	2.08 s	1.77 s	2.25 s	1.91 s
Dragon curve	(5, 2)	(3, 1)	5	2.05 s	1.44 s	1.84 s	1.22 s	1.85 s	1.23 s
Koch curve	(6, 4)	(6, 4)	20	28.6 s	20.2 s	16.6 s	8.30 s	16.8 s	8.13 s

Table 5: The deepening on the calibration experiments, seed 1, up to one clause more than the paper's. Paper: the paper's clause limit and `max_invented`; Found: the first candidate whose hypothesis is correct; Tried: the candidates run. Times are CPU totals over the candidates, of `learn/7` and of its generalisation step. [a]Stopped at $(5, 3)$ by a table-space limit. [b]Stopped at $(5, 3)$ by the 300 s limit. The Hilbert curve and the Sierpinski triangle were not run: their calibration experiments have many long examples without reuse (123 for the Hilbert curve), and each of Poker's candidates near the paper's limits takes minutes, more than this benchmark's budget of one process at a time allowed.

Problem	Sub-hypotheses	Poker		Forest	
		time	inferences	time	inferences
a^1b^1, $K = 1$	1	1.3 ms	6,164	0.2 ms	1,815
a^2b^2, $K = 3$	4	9.5 ms	199,910	0.9 ms	12,584
a^2b^2, $K = 5$	1,449	1.69 s	35,884,088	73 ms	1,911,052
a^3b^3, $K = 5$	10	1.19 s	29,254,138	2.7 ms	41,624
a^3b^3, $K = 7$	9,941	216 s	5,348,394,311	709 ms	17,141,301
a^4b^4, $K = 7$	18	200 s	5,134,352,920	7.2 ms	105,103
parity, $K = 5$	362	458 ms	4,971,369	38 ms	674,886

Table 6: A scaling test of the generalisation step: Poker's a^nb^n experiment in Vanilla at several sizes, without reuse, and its parity experiment (16 examples). Outputs identical in every row. Single runs of `compare_with_poker.pl`, wall-clock times; the inference counts are exact.

Problem (reuse)	Sub-hypotheses	Poker	Forest
hello world, $K = 2$	18	4.9 ms	0.9 ms
C–GNF a^nb^n, $K = 3$	18	56 ms	26 ms
C–GNF a^nb^m, $K = 4$	70	109 ms	6.9 ms
C–GNF even parity, $K = 5$	508	3.62 s	111 ms
C–GNF parentheses, $K = 4$	203	1.73 s	860 ms

Table 7: Poker against the forest with hypothesis reuse, on Poker's hello world and the settings of its Chomsky–Greibach script, with Poker's own unfolding of invented predicates applied to both. Outputs identical; single runs, wall-clock.

Task	Sub-hypotheses	Poker	Forest
reverse string	18,959	88.2 s	854 ms
parity check	21,740	32.3 s	1.32 s
even pairs, C–GNF	783	4.79 s	73 ms
reverse string, C–GNF	183	654 ms	29 ms
cycle navigation, C–GNF	266	371 ms	46 ms
parity check, C–GNF	227	194 ms	28 ms
robots, linear (Popper)	35	8.84 s	4.4 ms
even and odd (Louise)	45	17.9 ms	2.8 ms
grandparent with invention (Popper)	55	10.9 ms	3.8 ms
points-to, Andersen (Popper)	22	7.3 ms	2.3 ms
list membership (Hopper)	2	3.4 ms	4.3 ms
ancestor (Popper)	3	1.5 ms	0.5 ms
Inverse probe	1	0.9 ms	0.2 ms
cycle navigation	1,581,602*	not in 200 s	8.6 s*
even pairs	1,125,904*	not in 200 s	3.7 s*
robots, recursion (Popper)	3*	not in 200 s	98 ms*

Table 8: Poker against the forest on task files from other benchmarks, each with its own clause limit and without reuse (single runs of `compare_with_poker.pl` with a 16 GB stack, wall-clock, the forest's time including Poker's post-processing). C–GNF: weak Chomsky–Greibach constraints. *Distinct supports and the time to compute them, before post-processing, from an earlier run of the forest alone.

Language	l	Poker, `generalise/3`	SWI forest, supports	CeTTa, supports
even parity	21	5.83 s	46.8 ms	4.55 ms
$a^n b^n$	21	1.69 s	1.28 s	186 ms
equal a, b	21	2.55 s	271 ms	13.8 ms
$a^n b^m$	15–20	1.43 s	142 ms	5.2 ms
palindrome	21	11.8 s	3.4 ms	2.5 ms
parentheses	21	3.71 s	4.55 s	79 ms

Table 9: The last iteration of each context-free range: medians over seeds 1 to 3 (Poker's over 1 to 5, from Table 2). Poker's time includes its post-processing; the forests' times are of the supports alone.

9 Scope

- Without hypothesis reuse the forest computes the graded recurrence of Section 2; with reuse it computes the bounded least fixpoint of Section 4. Both laws are proved for ground derivation forests; that the depth-first invention state makes the forest's items correspond to Vanilla's refutations is checked against Poker, not proved.
- The forest replaces the generalisation step only. Poker's other stages are unchanged, and they still receive every sub-hypothesis as a list; they could work on the forest itself, which represents the same family compactly. On the L-systems with many generated examples these stages take most of the time (Table 3).
- With reuse, the forest's per-goal families are not limited by the budget that the rest of a refutation leaves, while Vanilla's search, which carries the metasubstitutions found so far, is. On long examples at small clause limits, and as `max_invented` grows, the forest's families grow faster than Poker's search (Section 8).

10 Running it

The forest is in `lib/forest`; tasks and scripts are in `data/forest_examples`. With Poker selected as the learner in `configuration.pl`, from Vanilla's root,

```
swipl data/forest_examples/compare_with_poker.pl TARGET EXPERIMENT [K]
```

compares the two `generalise/3` on an experiment file, for example with TARGET `s/2` and EXPERIMENT `data/poker_examples/anbn_poker.pl`. In the same directory,

- `poker_comparison.pl [N [K]]` does so on $a^N b^N$ at any size;
- `random_problems.sh` runs the 80 random problems;
- `benchmarks/survey/run_survey.sh` runs the task files of Table 8 and Poker's settings with reuse.

The AAAI-26 benchmark needs a clone of the paper's code appendix:

```
data/forest_examples/benchmarks/aaai26/setup_aaai26.sh CLONE WORK
data/forest_examples/benchmarks/aaai26/run_aaai26.sh WORK OUT 1 gen
```

and likewise `learn`, `readoff` and `tune`; `summarise.pl` tabulates the rows. In a Prolog session, `use_forest` makes Poker's `learn/1` use the forest. The forest was first written in MeTTa, where each memo table is a space of shared immutable terms indexed by their arguments; the files here are its port to SWI-Prolog, with the memo tables as asserted facts keyed by a variant hash or as tabled predicates.

References

[1] P. Buneman, S. Khanna and W.-C. Tan. Why and where: a characterization of data provenance. ICDT 2001.

[2] T. J. Green, G. Karvounarakis and V. Tannen. Provenance semirings. PODS 2007.

[3] S. Patsantzis. Vanilla: an inductive logic programming framework, with the learners Louise and Poker. `https://github.com/stassa/vanilla`.

[4] S. Patsantzis and S. H. Muggleton. Top program construction and reduction for polynomial time meta-interpretive learning. Machine Learning 110, 2021.

[5] G. Delétang, A. Ruoss, J. Grau-Moya, T. Genewein, L. K. Wenliang, E. Catt, C. Cundy, M. Hutter, S. Legg, J. Veness and P. A. Ortega. Neural networks and the Chomsky hierarchy. ICLR 2023.

[6] A. Cropper and R. Morel. Learning programs by learning from failures. Machine Learning 110, 2021.

[7] S. J. Purgał, D. M. Cerna and C. Kaliszyk. Learning higher-order logic programs from failures. IJCAI 2022.

[8] S. Patsantzis. Self-supervised inductive logic programming. In *Proceedings of the 40th AAAI Conference on Artificial Intelligence (AAAI-26)*, pages 19337–19344, 2026.

[9] S. Patsantzis. Code appendix of *Self-supervised inductive logic programming*. `https://github.com/stassa/aaai_26_experiments`, commit a446fca.

[10] J. Trewern, S. Patsantzis and A. Tamaddoni-Nezhad. Meta-interpretive learning as second-order resolution. IJCLR 2024.

[11] P. Prusinkiewicz and A. Lindenmayer. *The Algorithmic Beauty of Plants*. Springer, 1990.

[12] J. Wielemaker, T. Schrijvers, M. Triska and T. Lager. SWI-Prolog. *Theory and Practice of Logic Programming* 12(1–2), 2012.